CZECH TECHNICAL UNIVERSITY IN PRAGUE
Faculty of Electrical Engineering
Department of Control Engineering

FOC Firmware for Integrated BLDC Driver

Bachelor's thesis

Mathias Palme

Supervisor
Ing. Krištof Pučejdl

2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Palme** Jméno: **Mathias** Osobní číslo: **516266**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Otevřená informatika**

Specializace: **Základy umělé inteligence a počítačových věd**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

FOC firmware pro integrovaný BLDC driver

Název bakalářské práce anglicky:

FOC Firmware for Integrated BLDC Driver

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Krištof Pučejdl katedra řídicí techniky FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **26.01.2026** Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Palme Mathias

Podpis studenta

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Palme** Jméno: **Mathias** Osobní číslo: **516266**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Otevřená informatika**

Specializace: **Základy umělé inteligence a počítačových věd**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

FOC firmware pro integrovaný BLDC driver

Název bakalářské práce anglicky:

FOC Firmware for Integrated BLDC Driver

Pokyny pro vypracování:

The goal of this thesis is developing and testing embedded software that implements Field Oriented Control (FOC) for Brushless DC (BLDC) motors on an existing integrated three-phase motor driver with an STM-32 type microcontroller.
- Familiarize yourself with principles of FOC algorithm and review the existing implementations that use similar electronic components.
- Familiarize yourself with the specific uFOC hardware components their function, and the overall board layout and schematic.
- Develop and implement a firmware allowing the uFOC driver to control a BLDC motor using the FOC principle. Implement control modes for commanding output Torque, Velocity, and Position, and communication protocol via CAN bus, allowing setting, commanding and reading driver status from different devices connected on the bus.
- Create a documentation for the firmware on the existing GitHub repository.
- Test the firmware and demonstrate the control via simple benchmark application (for example a simple haptic device).

Seznam doporučené literatury:

[1] R. Gabriel, W. Leonhard and C. J. Nordby, „Field-Oriented Control of a Standard AC Motor Using Microprocessors,“ in IEEE Transactions on Industry Applications, 1980, doi: 10.1109/TIA.1980.4503770.
[2] A. Skuric et al., „SimpleFOC: A Field Oriented Control (FOC) Library for Controlling Brushless Direct Current (BLDC) and Stepper Motors,“ in Journal of Open Source Software, 2022, doi: 10.21105/joss.04232.
[3] R. C. Prayogo, A. Triwiyatno and M. A. Riyadi, „Field Oriented Control Implementation on BLDC Motor Controller with PI and SVPWM using STM32F103C8T6,“ in Journal of Physics: Conference Series (Vol. 2622). Institute of Physics, 2023, doi:10.1088/1742-6596/2622/1/012025

Declaration

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Mathias Palme
May 2026

Acknowledgments

Acknowledgment section need not to be too strict. Try to express your personality, sense of humor and so on :).

Abstract

The goal of this thesis is developing and testing embedded software that implements Field Oriented Control (FOC) for Brushless DC (BLDC) motors on an existing integrated three-phase motor driver called uFOC (micro FOC) with an STM-32 type microcontroller developed by Bc. Šimon Pecháček and Ing. Krištof Pučejdl at the Department of Control Engineering, Faculty of Electrical Engineering, Czech Technical University in Prague. The firmware implements FOC control algorithm to control torque. On top of torque control a PI velocity and PID position control loop is implemented. Aditionally communication via CAN bus is implemented with daisy chaining of multiple drivers, to allow indepemendent control and sensor readings from each driver via a signle CAN bus line.

Keywords: FOC, BLDC, PMSM, embedded software, motor control, electronic commutation

Abstrakt

Česká verze abstraktu (anotace) s klíčovými slovy na konci. Doplnit českou verzi...

Klíčová slova: antikolizní systém, digitální mapa, kalmanův filtr, V2X-komunikace.

Contents

1 | Introduction

BLDC and PMSM motors are increasingly more popular in varous applications, such as robotics, drones, electric vehicles and many more, due to their high efficiency, ability to produce large torque at low speeds and high position accuracy. Unlike brushed DC or universal motors, BLDC motors need specialized driver and control to achieve commutation.

While commercial BLDC drivers are available on todays market, such as ODrive Micro, moteus-c1, they are often expensive in range of 70-90 USD per driver. [9] [10] Therefore, the Department of Control Engineering at the Czech Technical University in Prague has developed an integrated three-phase motor driver called uFOC (micro FOC) with an STM-32 type microcontroller, which is a low-cost alternative to commercial drivers at BOM price of approximately 20 USD. [15]

The goal of the uFOC project was to develop a compact integrated motor driver for dynamic servo actuators, containing all components required for closed-loop torque control on a single printed circuit board. The concept of tightly integrating the motor, driver electronics, and communication interfaces into a compact actuator unit was inspired by the modular actuator design presented by Benjamin Katz in his work on low-cost actuators for dynamic robots. [5] The current revision of the uFOC driver has dimensions of 30×30 mm, approaching the practical physical limits imposed by components such as connectors. The driver is intended for direct mounting onto the motor assembly, minimizing external wiring requirements to only power and CAN bus connections. Two CAN connectors are provided to support daisy-chain communication in multi-actuator systems. [15]

To achive an efficient commutation and generate torque, the driver needs to implement a closed loop control. The most common control method for BLDC motors is Field Oriented Control (FOC), which allows for precise control of the motor's torque, speed and position by controlling the current in the stator windings.

Goal of this thesis is to develop embedded firmware that implements FOC current control, speed and position control loops and communication via CAN bus for the uFOC driver. Additionally a Python client side example for CAN communication with the driver is provided.

2 | Brief explanation of FOC

2.1 FOC overview

Field Oriented Control (FOC) is a closed loop control method for controlling torque of BLDC motors based on Clarke-Park Transformation. Standard BLDC motor is aquipped with three stator windings, which are supplied with three phase currents. To achive commutation and generate torque, propper voltage has to be provided to the stator windings by reading the phase currents (i_a, i_b, i_c). Three phase currents reference frames can be decomposed into two orthogonal torque and flux reference frames using Clarke transformation, resulting in two currents (i_α, i_β) in a stationary reference frame.

Park transformation is then applied to transform the currents from a stationary reference frame to a rotating reference frame, resulting in two DC currents (i_d, i_q), where i_d is a flux current and i_q is a torque current.

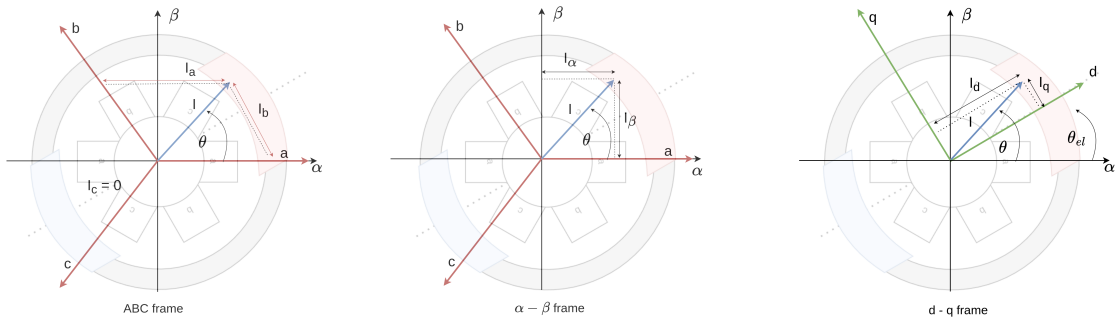

Figure 2.1: Coordinate Transformations in FOC. [13]

The diagram in Figure 2.1 shows the principle of Clarke and Park transformations used in FOC. Three phase stator currents are first projected from the (a, b, c) coordinate system into the stationary (α, β) frame using Clarke transformation. The stationary vectors are then rotated by the electrical rotor angle θ_e into the rotating (d, q) reference frame using Park transformation. This decomposition allows independent control of flux current i_d and torque current i_q.

To control i_d and i_q currents, two independent proportional-integral (PI) controllers are used to update applied voltage to the stator windings based on the error between the reference and the measured currents. In this implementation only the i_q current reference is changed to control the torque, while the i_d current reference is always set to zero to maximize the torque per ampere.

After getting new voltage references from the PI controllers, they need to be transformed back

to three phase voltages using inverse Park and inverse Clarke transformations, which can be then applied to the stator windings.

However applying the voltages directly from the inverse Park-Clarke transformations will achieve only aproximatelly 85 % of the maximum voltage difference between the stator windings, which is not optimal. [8] To maximize the voltage difference between the stator windings, Space Vector Modulation (SVPWM) can be applied to the voltage references.

Now that we can control the torque of the motor by controlling the i_q current reference, a velocity control loop can be implemented on top of the torque control by using a PI controller to update the i_q current reference based on the velocity error.

Aditionally, a position control loop can be implemented on top of the velocity control loop by using a PID controller to update the velocity reference based on the position error.

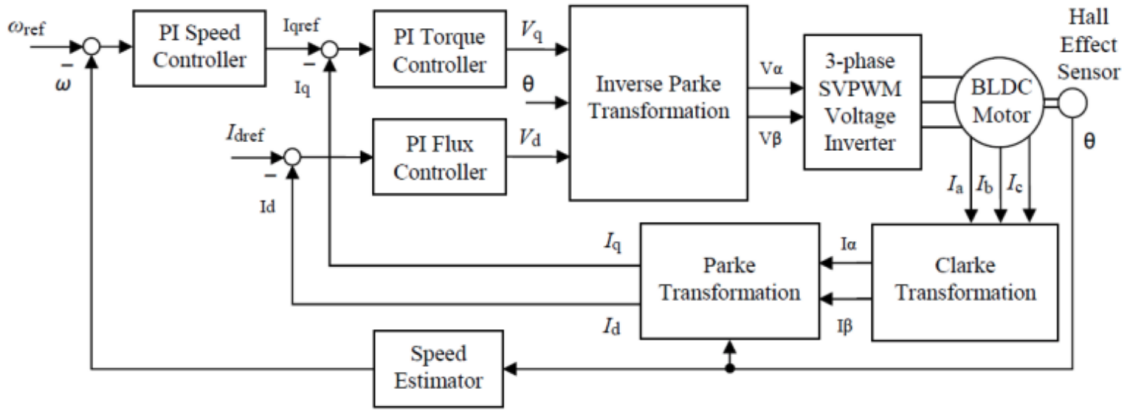

Figure 2.2: Field Oriented Control block diagram. [1]

The Figure 2.2 shows the cascaded structure of the FOC control system.

2.2 Clarke transformation

Clarke transformation is used to transform three phase currents (i_a, i_b, i_c) into two orthogonal currents (i_α, i_β) in a stationary reference frame, where i_α is torque generating current and i_β is flux producing current.

Clarke transformation can be represented as a matrix multiplication of the three phase currents with the transformation matrix

$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix}. \tag{2.1}$$

The scaling factor $\frac{2}{3}$ ensures amplitude invariance between the three-phase and i_α i_β representa-

tions.

The Figure 2.3 shows a Clarke transformation of 3-phase sinusoidal current into two orhogonal currents.

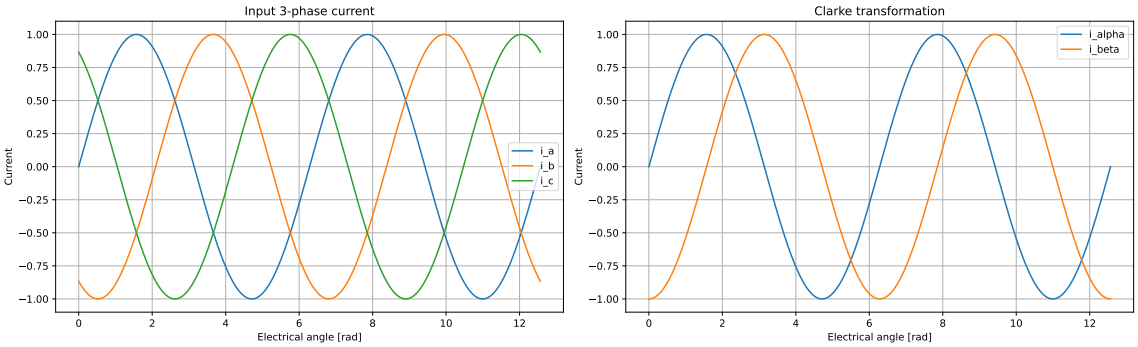

Figure 2.3: Clarke transformation of three-phase current.

.

Inverse Clarke transformation in context of FOC is used to transform two orthogonal voltages (v_α, v_β) in a stationary reference frame back to three phase voltages (v_a, v_b, v_c) that can be applied to the stator windings and can be expressed as

$$\begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} v_\alpha \\ v_\beta \end{bmatrix}. \tag{2.2}$$

2.3 Park transformation

Park transformation is used to transform two orthogonal currents (i_α, i_β) in a stationary reference frame from the stator point of view into two DC currents (i_d, i_q) in a rotating dq reference frame from a rotor point of view, where i_d is a flux current and i_q is a torque current. To perfrom Park transformation, the electrical angle θ_e must to be calculated based on the rotor position, which can be obtained from the encoder reading.

Park transformation can be represented as a matrix multiplication of the two orthogonal currents (i_α, i_β) with the transformation matrix

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \begin{bmatrix} \cos(\theta_e) & \sin(\theta_e) \\ \sin(-\theta_e) & \cos(\theta_e) \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix}. \tag{2.3}$$

Figure 2.4 shows the Park transformation applied to the output of the Clarke transformation shown in Figure 2.3, resulting in two DC current components.

In context of FOC, inverse Park transformation is used to transform two orthogonal DC voltages

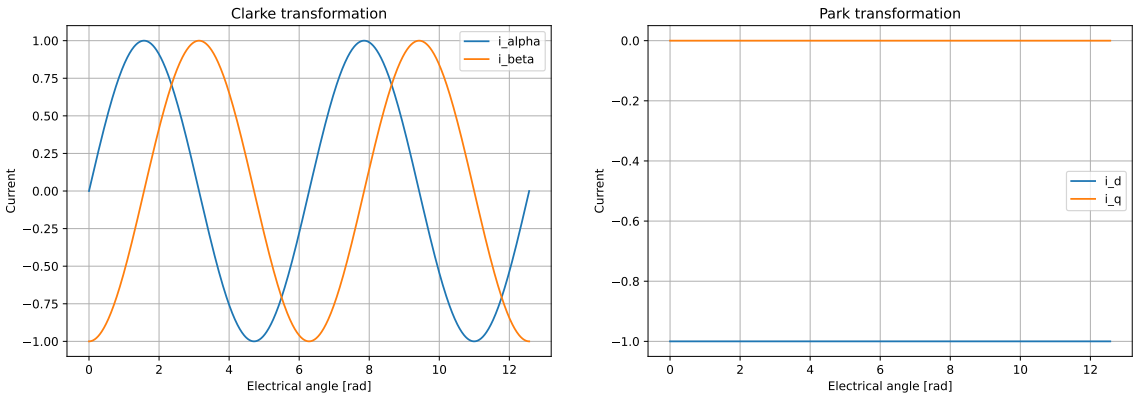

Figure 2.4: Park transformation of Clarke transformation output

(v_d, v_q), which are the outputs of the PI controllers, from a rotating reference frame back to two orthogonal voltages (v_α, v_β) in a stationary reference frame.

Inverse park transformation can be represented as a matrix multiplication of the two orthogonal DC voltages (v_d, v_q) with the transformation matrix

$$\begin{bmatrix} v_\alpha \\ v_\beta \end{bmatrix} = \begin{bmatrix} \cos(\theta_e) & -\sin(\theta_e) \\ \sin(\theta_e) & \cos(\theta_e) \end{bmatrix} \begin{bmatrix} v_d \\ v_q \end{bmatrix}. \tag{2.4}$$

2.4 PI and PID regulators

Proportional-Integral (PI) controllers are used in the FOC algorithm to regulate the d-axis and q-axis currents by adjusting the corresponding voltages (v_d, v_q). To prevent integrator windup during saturation conditions, anti-windup protection is implemented by limiting the integration process. The discrete-time PI controller with anti-windup can be expressed as

$$x_{out}(k+1) = K_p x_{in}(k) + w_{up} K_i T_s x_{in}(k) + x_{out}(k), \tag{2.5}$$

where

$$w_{up} = \begin{cases} 1, & \text{if } -x_{lim} < x_{in}(k) < x_{lim} \\ 0, & \text{otherwise} \end{cases} \tag{2.6}$$

and x_{lim} is the integral component limit. [2]

The proportional term provides immediate response to the control error, while the integral term eliminates steady-state error. In the implemented cascaded control structure, PI controllers are

used for current and velocity regulation.

For position control, a Proportional-Integral-Derivative (PID) controller is additionally used to improve damping and reduce overshoot. The PID controller generates the target angular velocity for the inner velocity control loop. [14]

2.5 Space vector modulation

Space Vector Pulse Width Modulation (SVPWM) is used to efficiently generate three-phase PWM signals for the inverter from the calculated voltage references. Compared to sinusoidal PWM, SVPWM provides better utilization of the DC bus voltage and allows approximately 15 % higher maximum output voltage. [8] After inverse Park and inverse Clarke transformations generate the three-phase voltage references (v_a, v_b, v_c), SVPWM computes the duty cycles for individual inverter phases and applies them to the MOSFET bridge.

The simplified SVPWM duty cycle equation can be expressed as

$$d_i = 0.5V_{dc} + v_{ref_i} + U^*, \tag{2.7}$$

where

$$U^* = -0.5 \left(\max \left(\sum_i v_{ref_i} \right) + \min \left(\sum_i v_{ref_i} \right) \right), \tag{2.8}$$

and v_{ref_i} represents the phase voltage references for phases a, b, and c. [2]

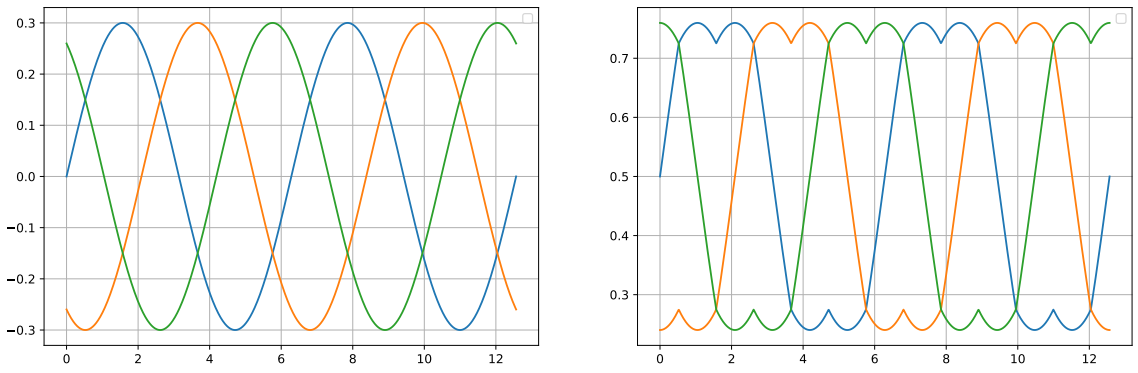

Figure 2.5: Modulation of a sinusoidal three-phase current using SVPWM.

2.6 Velocity and position control loops

The FOC torque control loop directly regulates the motor torque through the i_q current component. Once torque control is established, higher-level control loops can be added on top of

the inner current loop in order to control additional mechanical quantities.

Angular velocity control can implemented using a cascaded control structure. [14] The measured motor velocity derived from encoder readings is compared with the target velocity, and the resulting error is processed by a PI controller. The output of the velocity controller generates the target torque-producing current i_q for the inner FOC current control loop. In this way, motor velocity is indirectly controlled through regulation of the generated motor torque.

Similarly, position control can be implemented as an additional outer control loop. The measured rotor position is compared with the target position, and the resulting position error is processed by a PID controller. The output of the position controller generates the target angular velocity for the velocity control loop.

This creates a cascaded control hierarchy consisting of:

- current control loop regulating motor torque,
- velocity control loop regulating angular velocity,
- position control loop regulating rotor position.

The cascaded structure allows simultaneous control of motor torque, angular velocity, and position while maintaining stable dynamic behaviour. Typically, the inner current loop operates at the highest update frequency, while the outer velocity and position loops operate at lower frequencies due to slower mechanical system dynamics.

2.7 Electrical angle calculation

While encoder reading provides the mechanical angle of the motor shaft, the FOC algorithm operates with the electrical angle. The number of electrical rotations per one mechanical rotation is directly proportional to the number of pole pairs in the rotor. Therefore, the electrical angle θ_e can be expressed as

$$\theta_e = P \cdot \theta_m \tag{2.9}$$

where P is the number of pole pairs and θ_m is the measured mechanical angle. [7]

3 | Firmware implementation

3.1 Used technologies and development tools

STM32CubeMX was used to simplify the initial project setup, while the HAL library was employed to facilitate peripheral configuration and management. The firmware was implemented in the C programming language. CAN communication testing was performed using a Makerbase CANable V2.0 USB interface together with a Python based client script built on the pycan library. * Also compilation should be mantioned *

Arduino IDE was used as a serial monitor for debugging purposes.

3.2 Firmware topology

The firmware is structured into reusable modules located in the `uFOC_lib` directory. Peripheral initialization and high-level application logic are implemented in `src/main.c`.

uFOC_lib contains the following modules:

- `driver` - handles DRV8316CRRGFR driver configuration and control, ADC current sensing and PWM generation.
- `encoder` - handles MT6835GT encoder SPI configuration, positional data reading, angular velocity calculation and signal filtering.
- `foc` - implements all necessary transformations, to perform FOC control as well as trigonometric functions optimizations and SVPWM calculations.
- `config` - is used to store and control global configuration parameters of the firmware. This module implements an abstraction layer to simplify the access to the configuration parameters and ensure that they are updated correctly.
- `pi_controller` - implements PI and PID controllers with anti-windup and output saturation.
- `communication` - manages CAN communication, including message parsing and sending.

The current control loop, together with current sampling and encoder reading, is executed in the ADC injected conversion complete interrupt, which is triggered by the `TIM1_CC4` event at a frequency of 8888 Hz. This ensures precise alignment between current measurement and the PWM cycle. The selected control loop frequency corresponds to the maximum rate at which the

current implementation of the FOC algorithm can be executed reliably. The main performance limitation is currently the SPI based encoder communication.

Higher-level control loops, namely velocity and position control, are executed in a separate timer interrupt (`TIM6`) at a lower frequency of 2000 Hz, reflecting the cascaded structure of the control system.

This separation of control loops follows common practice in motor control systems, where fast inner loops regulate currents and slower outer loops regulate mechanical quantities.

3.3 MCU configuration

The uFOC driver uses an STM32F3xx microcontroller as the main processing unit. The MCU is responsible for real-time motor control, peripheral coordination, and communication with external devices. The configuration of the microcontroller is designed to meet the deterministic timing and low latency requirements of FOC.

3.3.1 System clock

The system clock is derived from the internal high-speed oscillator (HSI) operating at 8 MHz. This signal is multiplied using a phase-locked loop (PLL) with a multiplication factor of 16, resulting in a system clock frequency of 64 MHz.

The AHB bus runs at the full system clock frequency (64 MHz), while the APB1 bus is prescaled by a factor of 2 to 32 MHz, and the APB2 bus operates at 64 MHz. This configuration satisfies the peripheral frequency constraints while maintaining maximum performance for time-critical components.

Particular attention is given to peripherals involved in motor control. The advanced timer TIM1 and the ADC are both clocked at 64 MHz, ensuring high timing resolution for PWM generation and precise synchronization of current sampling.

This clock distribution enables deterministic timing of control loops and supports the strict real-time requirements of the control algorithm.

TIM6 is clocked from the APB1 timer clock. Although the APB1 peripheral clock is prescaled to 32 MHz, the timer clock is doubled to 64 MHz due to the APB1 prescaler being different from one. TIM6 is configured with a prescaler of 31 and an auto-reload value of 999, resulting in an update interrupt frequency of 2 kHz.

3.3.2 Timer configuration

The advanced timer TIM1 is used for PWM generation. It is configured in center aligned mode, which reduces switching harmonics and is commonly used in motor control applications. The timer operates at a PWM frequency of 8888 Hz.

TIM1 also generates a compare event (`CC4`) that is used to trigger ADC injected conversions. This ensures that current sampling is synchronized with the PWM cycle and occurs at a well-defined point in time.

A separate basic timer, TIM6, is used to periodically execute higher-level control loops. TIM6 operates at a frequency of 2 kHz and triggers an interrupt used for velocity and position control. This lower update rate reflects the cascaded structure of the control system, where slower outer loops are executed less frequently than the inner current control loop.

3.3.3 ADC configuration

The ADC is configured to perform injected conversions of three phase currents. The conversions are triggered by a TIM1 compare event, ensuring that current sampling occurs at a well-defined point within the PWM cycle. This synchronization minimizes measurement noise and improves control accuracy.

The ADC conversion complete interrupt is used to process the measured currents and execute the current control loop.

3.3.4 PWM ADC synchronization

In order to ensures deterministic timing and minimize control latency a precise synchronization between PWM generation and current measurement is achieved by using a timer-generated trigger for ADC conversions. The TIM1 compare event (`CC4`) initiates the ADC injected sequence, and the corresponding interrupt is used to execute the control algorithm.

The trigger signal is generated by a dedicated compare channel that is not used for PWM output, allowing independent adjustment of the sampling point within the PWM cycle.

3.3.5 Communication interfaces

The MCU communicates with external systems primarily via the CAN interface, which is used for command exchange and system control. The CAN peripheral is configured in normal mode with automatic retransmission enabled.

The CAN peripheral is clocked from the APB1 bus operating at 32 MHz. The bit timing parameters are configured to achieve a communication speed of 500 kbit/s. This interface is used for command exchange and system control and operates independently of the real-time control loops.

A UART interface is also available and is used only for debugging and diagnostic output during development.

SPI communication is used to interface with external peripherals, namely the magnetic encoder and the gate driver. Both devices share a single SPI peripheral, with separate chip-select signals used to access each device.

The SPI operates in master mode, and its configuration (clock polarity and phase) is adjusted as required by the connected peripherals.

The magnetic encoder is accessed within the real-time control loop to obtain rotor position information. The encoder update is performed inside the ADC interrupt service routine, ensuring that position data is synchronized with current measurements and control execution. As the SPI communication is performed in a blocking manner, its latency directly affects the execution time of the control loop.

In contrast, communication with the gate driver is primarily used for configuration. These operations are not time-critical and are performed outside the real-time control loop only during firmware initialization.

The SPI peripheral operates at a clock frequency of approximately 32 MHz, configured using a prescaler of 2. This high communication speed minimizes the duration of blocking SPI transactions, which is particularly important as encoder data acquisition is performed within the interrupt service routine.

3.4 Encoder

The rotor position is measured using an MT6835 magnetic encoder with a 21-bit absolute resolution and SPI communication interface.[12] The encoder interface is implemented as a reusable module located in `uFOC_lib/encoder`. This module provides encoder initialization, raw position acquisition, CRC verification, position unwrapping, velocity estimation, signal filtering and electrical offset calibration.

The encoder is accessed through the SPI3 peripheral. Since the SPI bus is shared with the gate driver, the encoder module explicitly configures the SPI peripheral before encoder communication. The encoder uses a dedicated chip-select signal, while the SPI transfer itself is performed in blocking mode. This is acceptable because the encoder read operation is executed inside the real-time control loop and must therefore be deterministic and short.

3.4.1 Encoder data structure

The encoder state is stored in the `encoder_t` structure. It contains the most recent raw encoder value, the previous raw value, an unwrapped absolute position counter, velocity estimates, filtering parameters, electrical angle information and diagnostic variables related to CRC checking.

The raw encoder position is stored as a 21-bit value in the range from 0 to $2^{21} - 1$. To handle continuous rotation across the encoder overflow boundary, the module computes the difference between consecutive samples and corrects it when the difference exceeds half of the encoder range. The corrected difference is accumulated into `position_ticks`, which represents the continuous mechanical position.

The structure also contains the `invert_dir` flag. This flag allows the logical direction of rotation to be inverted without modifying the raw encoder data. It affects the sign of the computed mechanical position, angular velocity and electrical angle.

Namely `encoder_t` contains:

- `bool invert_dir` – logical inversion of rotation direction which affects the sign of computed angle and velocity without modifying raw encoder data
- `uint32_t current_raw_value` – latest 21-bit raw position value read from the encoder via SPI
- `uint32_t prevous_raw_value` – previous raw position value, used to compute the delta between consecutive samples
- `float angular_velocity` – instantaneous angular velocity estimate (in rotations per second), computed from the filtered position increment
- `float angular_velocity_ewma` – exponentially weighted moving average (EWMA) of angular velocity which is used for velocity control to reduce signal noise
- `float angular_velocity_ewma_alpha` – EWMA coefficient α for velocity filtering, defines the trade-off between responsiveness and noise suppression
- `float ewma_value` – EWMA filtered absolute position value derived from `position_ticks`
- `float ewma_previous_value` – previous EWMA position value used for computing the filtered position difference
- `float ewma_delta` – difference between consecutive EWMA position values, serves as the basis for velocity estimation
- `float ewma_alpha` – EWMA coefficient α for position filtering, higher values reduce smoothing and improve responsiveness
- `int64_t position_ticks` – unwrapped absolute position in encoder ticks, accumulates deltas across revolutions without overflow
- `int32_t last_delta` – most recent position difference between samples
- `uint8_t valid` – data validity flag (currently not actively used)
- `uint8_t magnetic_pole_pairs` – number of motor pole pairs used to convert mechanical position to electrical angle
- `uint32_t electrical_offset` – calibrated offset between mechanical zero and electrical zero (obtained during calibration)
- `uint32_t electrical_angle_raw` – electrical angle expressed in raw encoder ticks (range 0 to 2^{21})

- `float electrical_angle` – electrical angle in radians which is used directly in FOC (Park/Clarke transforms)
- `uint32_t last_good_raw` – last valid raw encoder value (CRC verified), used as a fallback in case of communication errors
- `uint32_t crc_error_count` – counter of CRC errors detected during SPI communication with the encoder

3.4.2 Encoder initialization

The encoder is initialized using the `init_encoder()` function. During initialization, the first raw encoder value is read and used to initialize both the current and previous position values. The absolute position counter, EWMA filters and velocity variables are initialized from this first measurement.

The function also stores the number of motor pole pairs and the initial electrical offset. These parameters are required for conversion from mechanical position to electrical angle, which is used by the FOC algorithm.

3.4.3 Data reading and CRC control

Raw encoder data is read using the `mt6835_read_raw21()` function. The function sends a burst-read command to the MT6835 and receives six bytes over SPI. The 21-bit angle value is reconstructed from the received data bytes containing `ANGLE[20:13]`, `ANGLE[12:5]` and `ANGLE[4:0]`.

The encoder frame also contains status bits and an 8-bit CRC value. The module implements CRC-8 verification using a polynomial of `0x07` and an initial value of `0x00`. If the CRC check fails, the error counter `crc_error_count` is incremented and the last valid encoder value is reused. This prevents corrupted SPI measurements from directly affecting the control loop.

3.4.4 Position update and overflow handling

The encoder state is updated by calling `update_encoder()`. This function reads a new raw position value, verifies its CRC and computes the difference from the previous sample.

Because the encoder position is cyclic, the difference calculation must handle overflow at the end of the 21-bit range. If the measured difference is larger than half of the encoder modulo, the function assumes that the position crossed the overflow boundary and corrects the delta accordingly. The corrected delta is then accumulated into `position_ticks`.

This approach makes it possible to track continuous mechanical position over multiple revolutions while still using an absolute single-turn encoder.

3.4.5 Velocity calculation and signal filtering

The instantaneous angular velocity is calculated from the filtered position difference. The position signal is first processed by an Exponentially Weighted Moving Average (EWMA) filter. The filtered position difference is then converted from encoder ticks per sample to revolutions per second using the known sampling period.

The encoder module uses two EWMA filters. The first one filters the absolute position used for velocity estimation. Velocity v_t is then esstimated by a simple differentiation

$$v_t = \frac{p_t - p_{t-1}}{dt}, \tag{3.1}$$

where p_t is the latest position sample after EWMA filtering p_{t-1} is the previous position sample after EWMA filtering and dt is time difference between samples determined by encoder sampeling period.

The second EWMA filter is then applied to the velocity esstimations. This reduces measurement noise and improves the stability of the velocity control loop.

The velocity returned by `get_angular_velocity()` is the filtered angular velocity estimate from filtered absolute positions. The direction inversion flag is also taken into account, so the returned value has the correct sign with respect to the configured motor direction.

3.4.6 EWMA filter

The exponentially weighted moving average (EWMA) is used in this implementation to filter noisy angle and angular velocity measurements. EWMA was chosen due to its low memory and computational requirements. Unlike a moving average, which requires storing `n` previous time series values, EWMA requires storing only one value. This value is then used to compute the new EWMA value and is subsequently overwritten. Therefore, EWMA represents a lightweight method for filtering signals on an embedded device with limited resources.

EWMA can be represented by the equation

$$EWMA_t = \alpha \cdot x_t + (1 - \alpha) \cdot EWMA_{t-1}, \tag{3.2}$$

where $\alpha \in (0, 1)$ is a user-defined weighting parameter, x_t is the current value of the series, and $EWMA_{t-1}$ is the previous filtered value. [3]

3.4.7 Electrical angle calculation

After the mechanical position is updated, the module calculates the electrical angle required by the FOC algorithm. First, the unwrapped mechanical position is reduced back into one mechanical revolution. This value is multiplied by the number of motor pole pairs to obtain the corresponding electrical position.

The calibrated electrical offset is then subtracted from this value. The result is wrapped into the encoder range and converted to radians:

$$\theta_e = \frac{2\pi \cdot \texttt{electrical\_angle\_raw}}{2^{21}} \tag{3.3}$$

The resulting electrical angle is stored in `electrical_angle` and is used directly by the Park and inverse Park transformations in the FOC control loop.

3.4.8 Electrical offset calibration

Accurate alignment between the mechanical position acquired by the encoder and the electrical angle required by the FOC algorithm is essential for correct motor operation. For this purpose, the encoder module provides the `calibrate_electrical_offset()` function.

This function is blocking and must only be executed when the motor is not operating under closed loop control. It is recommended to perform the calibration with the motor unloaded. In a typical workflow, the calibration should be executed only once, the resulting electrical offset is read via the CAN interface, stored and applied during normal operation either by client side updating the electrical offset via can interface or recompiling the firmware with adjusted default `ENCODER_ELECTRICAL_OFFSET` in `config.h` without repeating the calibration during firmware initialization.

At the start of the calibration procedure, the ADC injected conversion interrupt is disabled, to stop the current control loop. A static voltage vector is then applied to the motor using the inverse Park and inverse Clarke transformations followed by SVPWM duty cycle computation. This forces the rotor into settle in a known and stable electrical position.

After a short settling period, a sequence of encoder samples is collected and averaged in order to reduce measurement noise. This value is then converted into the corresponding electrical position by accounting for the number of motor pole pairs:

$$\theta_{\text{e\_raw}} = \left(\theta_{\text{m\_raw}} \cdot p\right) \bmod 2^{21} \tag{3.4}$$

where $\theta_{\text{m\_raw}}$ represents averaged mechanical position in encoder ticks and p represents number of motor pole pairs.

The computed value $\theta_{\text{e\_raw}}$ in a known mechanical position is then used as the electrical offset θ_{offset} between the encoder mechanical zero and the actual electrical zero of the motor.

This offset is then used during normal operation to compute the electrical angle in radians as:

$$\theta_e = \frac{2\pi}{2^{21}} \left(\theta_{\text{m\_raw}} \cdot p - \theta_{\text{offset}}\right) \tag{3.5}$$

and is a crucial input to the FOC algorithm.

Once the calibration procedure is complete, the PWM outputs are returned to a neutral duty cycle and the ADC interrupt is re-enabled, restoring the standard control loop execution. The calibrated offset is then consistently applied during runtime, ensuring correct phase alignment required for stable FOC operation.

3.5 Gate motor driver

To control and measure currents in motor windings, a DRV8316C three-phase BLDC motor driver with built-in current sensing is used. The driver provides a configurable 6x or 3x PWM control scheme. In this implementation, the 6 PWM mode is used, which allows full control over both high-side and low-side transistors, enabling the implementation of advanced modulation strategies such as SVPWM and optimized current sensing.

Communication with the gate driver is implemented via an SPI interface, which is shared with the magnetic encoder. Since both peripherals require different SPI configurations (clock polarity and phase), the SPI peripheral is reconfigured before each transaction depending on the selected device. Each device is accessed using a dedicated chip-select signal, ensuring that only one peripheral is active on the bus at a time.

Unlike communication with the magnetic encoder, communication with the motor driver is not time-critical and is performed only during firmware initialization. As a result, it does not interfere with the time-critical execution of the FOC control loop.

The DRV8316C interface is implemented as a reusable module located in `uFOC_lib/driver`. This module encapsulates all low-level interactions with the motor driver, including SPI communication and device configuration. It also provides an abstraction for current sensing and PWM generation, allowing the higher-level control logic to remain independent of the specific hardware implementation. By isolating driver-specific functionality into a dedicated module, the firmware maintains a modular structure and can be more easily adapted to different motor drivers if required. [**?**]

3.5.1 PWM generation

PWM generation for the three motor phases is handled using the advanced timer TIM1. Since the DRV8316C driver is operated in 6 PWM mode, three main PWM outputs and three complementary PWM outputs are used to control the high-side and low-side transistors of the three-phase bridge.

The PWM outputs are started by the `pwm_init()` function, which enables all three PWM channels together with their complementary outputs. The duty cycles are updated using the `pwm_set()` function. This function receives normalized duty cycle values for phases A, B and

C in the range from 0 to 1. Before writing the values to the timer compare registers, the duty cycles are limited to this valid range in order to prevent invalid PWM states.

The normalized duty cycle values are converted to timer compare values according to the auto-reload value of TIM1. The resulting compare values are then written to the corresponding timer channels, which directly determine the switching duty cycle of each motor phase. This abstraction allows the FOC algorithm to work with normalized phase voltage commands without directly accessing timer registers.

In normal operation, the duty cycle values are produced by the SVPWM algorithm and passed to the driver module as normalized phase commands.

3.5.2 SPI communication and driver configuration

The DRV8316C is configured through the SPI interface. Since the SPI peripheral is shared with the magnetic encoder, the driver module first configures SPI3 to match the communication mode required by the gate driver before each transaction. For the DRV8316C, SPI is configured with low clock polarity and data sampling on the second clock edge.

Communication with the driver is implemented using 16-bit SPI frames. Each frame contains a read/write bit, a 6-bit register address, an 8-bit data field and a parity bit. The parity bit is calculated from the remaining frame bits before transmission and is used by the driver to detect communication errors.

The driver initialization is performed by the `drv8316_init()` function. At the beginning of the initialization sequence, the driver chip-select signal is disabled followed by a short delay to ensure a stable idle state of the SPI bus. The driver registers are then unlocked and the required configuration values are written to the control registers.

The `CTRL2` register is configured to clear existing fault flags, select the 6 PWM control mode, set the output slew rate to 50 V/μs and enable the SDO mode required for SPI readback. The `CTRL5` register is used to configure the gain of the integrated current sense amplifiers according to the selected `drv8316_csa_gain_t` value.

This value is passed as an argument to the `drv8316_init()` function and should be selected by the user based on the used motor and expected current draw. An enum structure `drv8316_csa_gain_t` was implemented to create an abstraction layer for the `drv8316_csa_gain_t` value and is later used for correct current conversion.

The `CTRL6` register configures the internal buck converter for a 3.3 V output. Setting the buck converter for a 3.3 V output is critical. Higher voltage will likely result in an instant MCU demidge.

3.5.3 Current sensing

Phase current measurement is implemented using the integrated current sense amplifiers (CSA) available in the DRV8316C motor driver.

Current sensing is configured during the driver initialization sequence through the `CTRL5` register, where the gain of the internal current sense amplifiers is selected using the `drv8316_csa_gain_t` enumeration. The selected gain directly affects the measurable current range and the current measurement resolution. Lower gain values allow measurement of higher currents, while higher gain values provide better resolution for lower current levels. The avialable gain configurations correspond to current conversion factors of 0.15, 0.3, 0.6 and 1.2 V/A.

The amplified current sense signals are connected to ADC inputs of the STM32 microcontroller. Current sampling is synchronized with the PWM timer in order to obtain measurements at deterministic points within the PWM cycle and minimize the influence of switching noise. The ADC conversions are triggered directly by the timer hardware, ensuring stable sampling timing independent of software execution latency.

The PWM timer operates in center-aligned mode, which ensures symmetrical switching of the inverter transistors during the PWM period. This creates a stable interval around the middle of the PWM cycle where the phase currents are minimally affected by switching transients. As a result, ADC sampling performed in this interval provides more accurate and repeatable current measurements.

The measured ADC values are converted to phase currents using the known ADC reference voltage, ADC resolution, shunt resistor value, and the configured CSA gain. Since the current sense amplifier outputs are offset to a reference voltage representing zero current, the measured ADC value must first be shifted by a calibrated offset value before the phase current is calculated.

The driver implements the `driver_current_offsets_t` structure to store calibrated offsets for each ADC channel separately. Since the ADC operates with 12-bit resolution, the `uint16_t` data type is used to store the offset values. Furthermore, the `driver_phase_currents_t` structure is implemented to store the latest current measurements for individual motor phases as floating-point values in amperes.

During normal operation of the FOC current control loop, the `driver_phase_currents_from_adc()` function is used to convert raw ADC measurements into phase currents. The function receives raw ADC samples together with the calibrated offsets and configured CSA gain, and stores the converted current values into the `driver_phase_currents_t` structure.

3.5.4 Current sensing offset calibration

The current sense amplifiers generate an output voltage centered around a predefined zero-current reference level, approximately in the middle of the ADC input range. Therefore, an

offset calibration procedure is performed during firmware initialization.

The driver module implements the `driver_current_calibrate_offsets()` function, which receives a pointer to the `driver_current_offsets_t` structure, pointers to variables containing raw ADC current samples, the number of calibration samples, and the delay between samples in milliseconds.

The function itself does not perform ADC sampling directly. Therefore, the raw ADC current values must be continuously updated by the TIM1 interrupt routine while the calibration procedure is running. It is also required that the PWM outputs are already initialized and all PWM channels are set to a duty cycle of 0.5, ensuring that no phase current flows through the motor windings during calibration. Failure to satisfy these conditions may result in incorrect offset calibration.

The calibration function averages the acquired ADC samples and stores the calculated offset values into the `driver_current_offsets_t` structure.

3.6 PI and PID regulators

Both PI and PID controllers are implemented in the `uFOC_lib/pi_controller` module. Each controller type defines its own data structure for storing controller parameters and internal state variables. Both controller implementations follow a consistent API naming convention.

3.6.1 PI controller

The PI controller is implemented using the `PIController` structure, which stores the controller gains, internal integration state, and output limitation parameters.

The structure contains the following variables:

- `float kp` – proportional gain
- `float ki` – integral gain
- `float integral` – accumulated integral term
- `float out_max` – maximum absolute output value

The controller is initialized using the `pi_init()` function, which configures the proportional and integral gains together with the maximum allowed output value. During initialization, the internal integral accumulator is set to zero.

The `pi_update()` function performs one iteration of the PI control algorithm. The function receives the current control error together with the elapsed time step `dt`. First, the proportional term is computed directly from the current error. Then a new integral value is calculated by integrating the error over time.

To prevent excessive integral accumulation during output saturation, an anti-windup mechanism is implemented. If the computed output exceeds the configured output limit, the output is clamped to the allowed range. The integral term is updated only when the current error would reduce the saturation state. This prevents uncontrolled growth of the integral component and improves regulator stability after saturation.

The `pi_set_out_max()` function allows modification of the maximum allowed controller output during runtime. The `pi_reset()` function clears the internal integral accumulator and restores the controller state.

3.6.2 PID controller

The PID controller extends the PI controller by adding a derivative component. It is implemented using the `PIDController` structure.

The structure contains the following variables:

- `float kp` – proportional gain
- `float ki` – integral gain
- `float kd` – derivative gain
- `float integral` – accumulated integral term
- `float prev_error` – previous control error
- `uint8_t has_prev_error` – flag indicating whether a valid previous error exists
- `float out_max` – maximum absolute output value

The controller is initialized using the `pid_init()` function, which configures all controller gains and initializes the internal state variables. The derivative term requires knowledge of the previous control error. For this reason, the controller stores both the previous error value and a flag indicating whether a valid previous error sample is available.

The `pid_update()` function performs one iteration of the PID control algorithm. The proportional and integral terms are calculated in the same way as in the PI controller. The derivative term is calculated as

$$d = k_d \frac{e_t - e_{t-1}}{dt} \ , \tag{3.6}$$

where e_t is the current control error and e_{t-1} is the previous error value.

The derivative term is only evaluated when a valid previous error sample exists and the time step is greater than zero. During the first controller iteration, the derivative component is therefore set to zero.

Similarly to the PI controller, the PID implementation also includes output saturation and

anti-windup protection. If the controller output exceeds the configured limit, the output is clamped while the integral term is only updated when the control error reduces the saturation condition.

The `pid_set_out_max()` function allows runtime modification of the maximum controller output. The `pid_reset()` function clears the integral accumulator, resets the stored previous error value, and invalidates the derivative history.

3.7 FOC implementation

The FOC module located in the `uFOC_lib/foc` directory implements the mathematical core of the field-oriented control algorithm. The module provides functions for the Clarke and Park transformations together with their inverse transformations, which are used for conversion between the three-phase stationary reference frame and the rotating dq reference frame.

The module also implements space vector PWM modulation through the `svpwm()` function, which converts the calculated phase voltage commands into normalized PWM duty cycles suitable for motor control.

To reduce the computational cost of trigonometric operations, sine and cosine functions are implemented using a lookup table with linear interpolation. [6] Compared to standard floating-point implementations from the `math.h` library, this approach significantly reduces control loop execution time.

The main FOC algorithm is encapsulated in the `foc_compute_voltages()` function, which performs coordinate transformations, current control calculations, and generation of output voltage commands.

3.7.1 Clarke and Park transformations

The Clarke and Park transformations are implemented by the `clarke_transform()` and `park_transform()` functions. Their inverse transformations are implemented by the `inv_clarke_transform()` and `inv_park_transform()` functions.

Optimizations were implemented in the Park and inverse Park transformations by using a precomputed look-up table for trigonometric functions. This optimization is further described in Chapter 3.7.2.

3.7.2 Trigonometric functions optimization

Trigonometric functions, namely sine and cosine, are used in the Park and inverse Park transformations. To reduce the computational cost and improve the execution speed of the FOC algorithm, a lookup table with linear interpolation was implemented for sine and cosine evaluation. [6]

During firmware initialization, the lookup table is generated using the `init_sin_table()` function. The number of samples stored in the lookup table is determined by the `SIN_TABLE_SIZE` constant defined in `foc.c`. The default number of values in this implementation was set to 128. The calculated values are stored in the `static float sin_table[SIN_TABLE_SIZE]` array.

Since the sine and cosine functions are related by

$$\cos(x) = \sin\left(x + \frac{\pi}{2}\right) \tag{3.7}$$

only a single sine lookup table is required. The cosine value is obtained by evaluating the sine function with a phase shift of $\frac{\pi}{2}$.

Because the Park and inverse Park transformations always require both sine and cosine values for the same angle, a combined `fast_sincos()` function was implemented. This function receives the electrical angle θ together with pointers to the output variables for the calculated sine and cosine values.

To improve accuracy, linear interpolation between neighbouring lookup table samples is used. Compared to repeated evaluation of the standard `math.h` trigonometric functions, this approach significantly reduces execution time and improves determinism of the real time control loop.

3.7.3 FOC torque control loop

In Field Oriented Control of a PMSM, the electromagnetic torque is directly proportional to the quadrature-axis current component i_q. [2] Therefore, the implemented firmware primarily referes to toqrue control loop as current control loop.

The FOC current control loop is executed inside the `HAL_ADCEx_InjectedConvCpltCallback()` callback function, which is triggered after completion of the timer synchronized injected ADC conversion. This approach ensures deterministic execution timing and proper synchronization between current sampling and PWM generation.

The control loop execution is divided into several steps. First, raw ADC samples from the current sense amplifiers are acquired and converted into phase currents expressed in amperes using the `driver_phase_currents_from_adc()` function from the `uFOC_lib/driver` module.

Next, the `update_encoder()` function located in the `uFOC_lib/encoder` module is called to obtain the updated rotor position and calculate the electrical angle required for the FOC transformations.

After the current measurements and rotor position are updated, the firmware checks the currently selected control mode represented by the `ControlState` enumeration. The FOC current control algorithm is executed only when the control state is configured for current, velocity, or position control operation.

If the FOC current control loop is enabled, the measured electrical angle, measured phase currents, and reference values for the torque and flux currents are passed to the `foc_compute_voltages()` function.

Inside this function, the measured phase currents are transformed into the rotating dq reference frame using the Clarke and Park transformations. The transformed current values are then compared with the reference current values, and the resulting current errors are processed by independent PI controllers for the d and q axes. The calculated voltage commands are subsequently transformed back into three-phase quantities using the inverse Park and inverse Clarke transformations.

The resulting phase voltage commands are then passed to the `svpwm()` function, which generates normalized voltage values suitable for PWM generation. Finally, the calculated PWM duty cycles are normalized with respect to `V_BUS_NOMINAL` and applied to the motor driver using the `pwm_set()` function.

3.7.4 Velocity and position control loops

In addition to the inner FOC toqrue control loop, the firmware also implements higher level velocity and position control loops. These control loops are implemented as cascaded controllers, where the output of the higher level controller serves as the reference input for the lower level controller.

Unlike the current control loop, which is synchronized with PWM generation and ADC sampling, the velocity and position control loops are executed independently inside the `HAL_TIM_PeriodElapsedCallback()` interrupt callback function triggered by `TIM6`. The control loops operate with a fixed sampling period of 0.5 ms corresponding to an update frequency of 2 kHz.

The velocity control loop is enabled when the configured `ControlState` is set to `VELOCITY_CONTROL` or higher. The current angular velocity filtered using EWMA is obtained from the encoder using the `get_angular_velocity()` function. The measured velocity is compared with the target velocity value and the resulting velocity error is processed by a PI controller implemented using the regulator module located in `uFOC_lib/pi_controller`.

A PI controller was selected instead of a full PID controller because experimental testing showed that the derivative component significantly amplified measurement noise present in the calculated velocity signal, resulting in unstable and noisy control behaviour. The controller output is used as the reference torque current for the inner FOC current control loop.

When the control mode is configured for position control, an additional outer position control loop is executed. The current rotor position is obtained using the `encoder_get_turns()` function and compared with the target position. The resulting position error is processed by a PID controller implemented using the same regulator module.

Unlike the velocity controller, the derivative component proved beneficial for position control, where it reduced overshoot and improved transient response during experimental testing. The controller output generates the target angular velocity for the velocity control loop.

3.8 Configuration abstraction layer

The firmware implements a configuration abstraction layer located in the `uFOC_lib/config` directory. This module centralizes storage and management of globally accessible configuration parameters used by the motor control firmware.

The main purpose of the configuration module is to provide a unified interface for handling runtime configurable variables, such as target torque current, target velocity, target position, current and velocity limits, and the currently selected control mode. These parameters are stored inside the `config_t` structure, which acts as the main shared configuration object used across the firmware.

In addition to runtime configuration variables, the module also defines firmware constants and default parameters, including motor specific configuration values such as the number of magnetic pole pairs, encoder electrical offset and initial regulator limits.

The module further implements multiple setter and getter functions used for safe access to configurable parameters. These functions are primarily intended for communication interfaces such as CAN bus control, but can also be used directly by the application firmware. Besides simplifying access to configuration variables, the abstraction layer also ensures that configurable values remain within predefined safety limits.

The configuration module additionally implements unit conversion utilities for angular quantities. Internally, all angular values are represented in rotations, while the abstraction layer allows the user to work with rotations, radians, or degrees. Changing the selected user units only affects the external interface of the configuration module, while all internal calculations and stored values remain represented in rotations.

3.8.1 Configuration constants

The configuration module defines multiple compile-time constants used to configure firmware behaviour, motor parameters, communication settings, and default controller gains. These constants are defined in `config.h` and are used during firmware initialization.

The implemented configuration constants include:

- `CAN_COMMUNICATION_DEVICE_ID` – device identifier used for CAN communication and daisy-chain addressing
- `CAN_COMMUNICATION_STD_ID` – standard CAN arbitration identifier used for communication

- `DEFAULT_USER_UNITS` – default angular units used by the configuration abstraction layer; the value must correspond to one of the options defined in the `AngleUnits` enumeration
- `MOTOR_MAGNETIC_PAIRS` – number of magnetic pole pairs of the motor rotor used for electrical angle calculation
- `ENCODER_ELECTRICAL_OFFSET` – default encoder electrical offset used for rotor position alignment
- `ENCODER_INVERT_DIR` – boolean value defining whether the encoder rotation direction is inverted
- `V_BUS_NOMINAL` – nominal DC bus voltage used for voltage normalization during PWM generation
- `PI_CURRENT_KP` – proportional gain of the current PI controllers shared by the d and q current control loops
- `PI_CURRENT_KI` – integral gain of the current PI controllers
- `PI_CURRENT_HARD_LIMIT` – maximum allowed motor phase current
- `PI_CURRENT_SOFT_LIMIT` – default software current limit used during normal operation
- `PI_VELOCITY_KP` – proportional gain of the angular velocity PI controller
- `PI_VELOCITY_KI` – integral gain of the angular velocity PI controller
- `PI_VELOCITY_HARD_LIMIT` – maximum allowed angular velocity expressed in rotations per second
- `PI_VELOCITY_SOFT_LIMIT` – default software angular velocity limit expressed in rotations per second
- `PI_POSITION_KP` – proportional gain of the position PID controller
- `PI_POSITION_KI` – integral gain of the position PID controller
- `PI_POSITION_KD` – derivative gain of the position PID controller

3.8.2 Config structure

The configuration layer implements the `config_t` structure, which encapsulates the shared configuration and runtime control variables used throughout the firmware.

To simplify handling of the implemented control regulators, an additional `regulators_t` substructure is used to encapsulate all instances of the `PIController` and `PIDController` structures. The `regulators_t` structure contains the following controller instances:

- `PIController pi_d_current_axis`

- `PIController pi_q_current_axis`
- `PIController pi_angular_velocity`
- `PIDController pid_position`

The main `config_t` structure stores the following configuration variables and runtime control parameters:

- `regulators_t regulators`
- `float torque_current_target`
- `float torque_current_soft_limit`
- `float flux_current_target`
- `float flux_current_soft_limit`
- `float angular_velocity_target`
- `float angular_velocity_soft_limit`
- `float position_target`
- `enum ControlState control_state`
- `enum AngleUnits user_angle_units`
- `encoder_t encoder`

The structure is initialized using the `init_config()` function. During initialization, all target values are set to zero and the default software limits are configured using the predefined configuration constants. The initialization function also initializes all PI and PID controller structures using the default controller gains and limits defined in the configuration module. Finally, the control state is set to `NO_CONTROL`.

3.8.3 Setters and getters

The configuration module provides a set of setter and getter functions used to access and modify selected configuration parameters. These functions form a controlled interface between the global configuration structure and other firmware modules, such as the CAN communication layer or the main application logic. All setter and getter functions receive a pointer to the `config_t` structure as their first argument.

The implemented functions can be divided into several groups. The first group contains functions related to the selected control mode. The `set_control_state()` function updates the active control state and is used to enable or disable the implemented control loops.

The second group contains functions for current control configuration. These functions allow modification of the torque current target, torque current software limit, and PI controller gains used by the inner current control loop.

The third group contains functions for angular velocity control. They allow setting the target angular velocity, changing the velocity software limit, reading the configured velocity limit, and modifying the gains of the angular velocity PI controller.

The position control functions provide access to the target position, measured position, and PID controller gains used by the outer position control loop. The measured position is obtained indirectly through the encoder instance stored inside the `config_t` structure.

The configuration module also provides functions for accessing and modifying the encoder electrical offset. This allows the electrical alignment value to be adjusted without directly accessing the encoder structure.

Finally, the module implements utility functions for user unit configuration and angle conversion. The `set_user_angle_units()` and `get_user_angle_units()` functions configure the angular units used by the external interface. Conversion functions are provided for converting between the internally used rotation representation and user-facing units, including rotations, degrees, and radians.

3.8.4 Control state

The currently active control mode is represented by the `ControlState` enumeration, which determines which control loops are enabled during firmware operation. The `ControlState` enumeration contains the following values:

- `NO_CONTROL` – all control loops are disabled
- `CURRENT_CONTROL` – only the inner FOC current control loop regulating the flux and torque currents is enabled
- `VELOCITY_CONTROL` – the velocity control loop together with the inner current control loop is enabled
- `POSITION_CONTROL` – the position, velocity, and current control loops are enabled

The enumeration values are intentionally ordered to reflect the cascaded structure of the implemented control system. This arrangement allows individual control loops to determine whether they should be active using simple comparison operations. For example, the velocity controller only needs to check whether the current control state is greater than or equal to `VELOCITY_CONTROL`.

The control state can be modified using the `set_control_state()` function. Setting the control state to `NO_CONTROL` disables all control loops and automatically sets all PWM outputs to a neutral duty cycle corresponding to zero motor voltage.

3.8.5 Limits configuration

The configuration abstraction layer implements hard and soft limits for selected control variables.

Hard limits represent the physical limitations of the motor and power electronics, such as the maximum allowed phase current or maximum angular velocity. These limits are defined as compile-time configuration constants and cannot be modified during runtime operation.

Soft limits are used to restrict the maximum output values used during normal operation of the control system. Unlike hard limits, soft limits can be modified during runtime using the implemented setter functions or through the CAN communication interface.

The implemented setter functions automatically ensure that configured soft limits cannot exceed the corresponding hard limits. If a user attempts to set a soft limit above the allowed hard limit, the value is automatically clamped to the corresponding hard limit value.

The same protection mechanism is also applied to target values such as target current, velocity, or position commands. Whenever a target value is updated using a setter function, the value is automatically limited to remain within the configured software limits. This prevents invalid control commands and improves operational safety of the motor control firmware.

3.8.6 Units conversion and configuration

The configuration module implements the `AngleUnits` enumeration used to store the currently selected user angle units. Although all internal angular quantities are represented in rotations, the abstraction layer allows the user to work with rotations, radians, or degrees.

Because of this abstraction, setter functions internally convert user selected angle values into rotations, while getter functions convert internally stored rotation values back into the configured user units.

To simplify these conversions, the configuration module implements the `convert_rotations_to_user_units()` and `convert_user_units_to_rotations()` functions.

The `convert_rotations_to_user_units()` function receives a pointer to the active `config_t` structure together with the value to convert expressed in rotations. Based on the currently configured `AngleUnits` value, the function converts the internal rotation value into the corresponding user units.

Similarly, the `convert_user_units_to_rotations()` function converts a value expressed in user selected units into the internal rotation representation used throughout the firmware.

These conversion functions are automatically used by all setter and getter functions working with angle values, namely position and angular velocty. As a result, the internal firmware implementation remains fully independent of the externally selected user units.

3.9 CAN communication

CAN communication is implemented in the `uFOC_lib/communication` module. The module provides an interface between external client connected to the CAN bus and the internal firmware configuration layer. It allows an external controller to change the active control mode, update reference values, modify controller parameters, read selected feedback values, and execute encoder calibration.

The communication module is divided into two main parts. The first part consists of `communication.h` and `communication.c`, which implement the low-level CAN interface. This includes CAN initialization, filter configuration, message transmission, message reception, and buffering of the last received frame.

The second part consists of `commands.h` and `commands.c`, which implement the command layer. This layer translates received CAN commands into calls of setter and getter functions from the `config` module.

A simple CAN message frame format is used. The first data byte contains the target device identifier, the second byte contains the command identifier, and the remaining bytes contain the command payload. This allows multiple uFOC drivers to share the same CAN bus while still being addressed independently.

3.9.1 CAN communication in general

The low-level CAN communication layer is implemented in `communication.c`. Before the CAN peripheral can be used, the function `communication_init()` is called with a pointer to the HAL CAN handle. This function stores the CAN handle internally and clears the receive buffer.

The CAN peripheral is started using the `communication_start()` function. This function first configures the CAN acceptance filter and then starts the CAN peripheral using `HAL_CAN_Start()`. After that, the RX FIFO 0 pending message interrupt is enabled using `HAL_CAN_ActivateNotification()`. The implemented filter accepts all incoming messages, while the actual device addressing is handled later in software by checking the device ID stored in the message payload.

Messages are transmitted using the `communication_send()` function. This function creates a standard CAN data frame, sets the standard arbitration identifier, configures the data length code, and sends the message using `HAL_CAN_AddTxMessage()`. Only standard CAN identifiers are used in this implementation.

Received messages are processed in the CAN receive interrupt callback. When a new message is available in RX FIFO 0, the function `communication_rx_fifo0_pending_callback()` reads the message using `HAL_CAN_GetRxMessage()` and stores it into an internal `can_message_t` structure. This structure contains the message identifier, data length, data bytes, and flags

specifying whether the frame uses an extended identifier or a remote frame.

The function `communication_message_available()` can be used to check whether a new message has been received. The function `communication_read()` receives a pointer to `can_message_t` structure and copies the received message from the internal buffer into the structure. Interrupts are temporarily disabled during this copy operation to prevent the receive buffer from being modified while it is being read.

3.9.2 Command handling

The command layer is implemented in `commands.c`. Its purpose is to parse received data and translate CAN commands into calls of the configuration abstraction layer. This keeps the communication code separated from the internal implementation of control variables and controller parameters.

To execute received CAN commands, the `handle_communication()` function was implemented. This function first reads the first byte of the data payload and compares it with the local device ID to determine whether the current device is the target recipient. If the IDs do not match, the command is ignored.

The second byte of the payload is then interpreted as the command identifier. The command identifiers are defined in `commands.h`. A `switch` statement is subsequently used to execute the corresponding command handler.

The implemented commands can be divided into setter commands, getter commands.

Setter commands first call the `get_float_value_from_message()` helper function with a pointer to the `can_message_t` structure. This function extracts bytes 2–5 from the CAN payload and converts them into a floating point value. The parsed value is then passed to the corresponding setter function implemented in the `config.c` module.

Getter commands first call the corresponding getter function from the `config.c` module. The returned value is then passed to the `assemble_data_with_float_value()` helper function, which assembles a CAN response frame. This function sets the first payload byte to `CAN_COMMUNICATION_DEVICE_ID`, the second byte to the corresponding command identifier, and fills bytes 2–5 with the requested floating point value. After the response message is assembled, the `communication_send()` function is called to transmit the response back to the client via the CAN bus.

The only exception is the `GET_USER_ANGLE_UNITS` command, which does not return a floating point value but an `AngleUnits` enumeration value. For this reason, a custom message assembly procedure was implemented specifically for this command.

3.10 Python CAN client example

An example Python client was implemented in `pytools/ufoc_client.py` using the `python-can` library. [11] The module implements the `uFOCclient` class, which provides a high-level API for communication with the uFOC driver through a Makerbase CANable interface.

The client implements helper functions for all commands defined in `commands.h`, including configuration of control modes, regulator parameters, target values, and reading feedback values from the driver.

The `ufoc_client.py` module was primarily used for firmware testing and validation of CAN communication functionality.

4 | Testing and results

To verify the functionality of the developed firmware, the individual control loops were tested first after PI and PID controller tuning. Afterwards, a practical usage example was demonstrated using CAN communication between a Python client and two uFOC drivers connected in a daisy-chain configuration.

For testing, two iPower GM4108 motors powered from a 20 V power supply were used.

4.1 Control loops testing

Testing of the individual control loops was performed by modifying the firmware main loop to periodically transmit the reference and measured variables over UART, while CAN communication was used to update the target values in real time.

4.1.1 Torque loop testing

The current control loop was tested by changing the target torque current from 0 A to 0.4 A, then to 0.75 A, and finally back to 0 A, while the motor shaft was physically blocked to allow the motor to generate maximum torque.

During the test, the target flux current was set to zero.

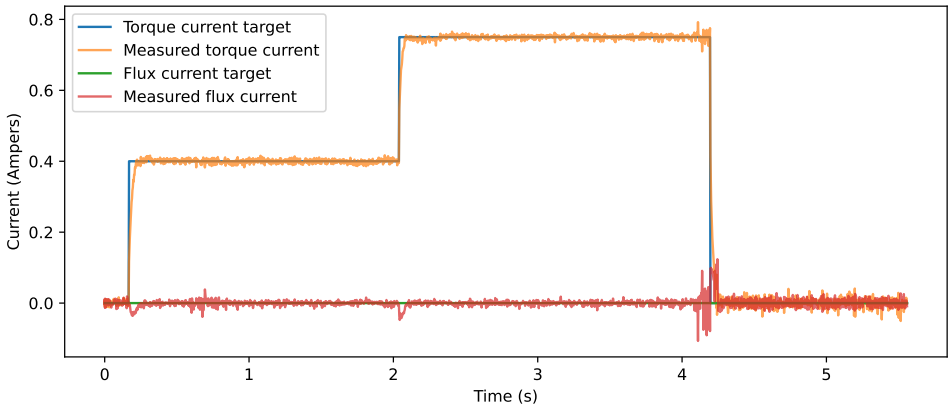

Figure 4.1: Torque control loop under load

Figure 4.1 shows both torque and flux current components together with their target and measured values. The measured torque current closely follows the commanded reference with minimal steady-state error and low noise. At the same time, the flux current regulator successfully

maintains the flux current near zero, with only brief transient deviations occurring during rapid changes of the torque current reference.

The results demonstrate stable operation of the implemented FOC current control loop and proper decoupling between the torque (i_q) and flux (i_d) current components.

4.1.2 Velocity loop testing

The velocity control loop was tested on an unloaded motor by changing the reference velocity from 0 rot/s to 4 rot/s, then to -8 rot/s to demonstrate operation in the opposite direction, and finally to 18 rot/s.

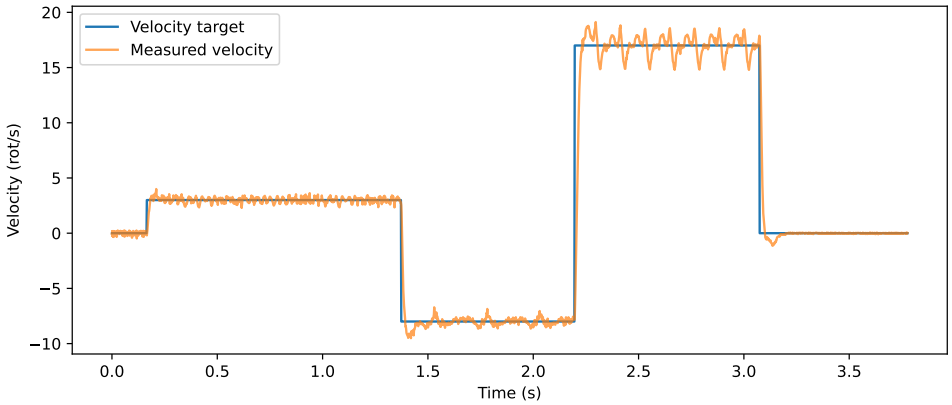

Figure 4.2: Velocity control

According to the manufacturer, the used GM4108 motor should achieve approximately 513–567 RPM, corresponding to 8.55–9.45 rot/s [4]. However, testing showed that significantly higher rotational speeds could be achieved using the uFOC driver and the implemented FOC algorithm.

Figure 4.2 shows that the controller is able to accurately track the reference velocity within the nominal operating range of the motor, including stable operation in both rotational directions. When operating above the rated motor speed, the controller was still able to accelerate the motor further, but noticeable oscillations appeared in the measured velocity signal. These oscillations are likely caused by limitations of the motor mechanical design and increased back-EMF at higher rotational speeds rather than instability of the control loop itself.

The results demonstrate correct functionality of the velocity controller and successful integration of the cascaded velocity and current control loops.

4.1.3 Position loop testing

The position control loop was tested on an unloaded motor by changing the target position from 0 to 3 rotations, then to 6 rotations, and finally back to 3 rotations.

Figure 4.3 demonstrates smooth and stable position tracking of the implemented controller. The measured position closely follows the target trajectory without significant steady-state error. A

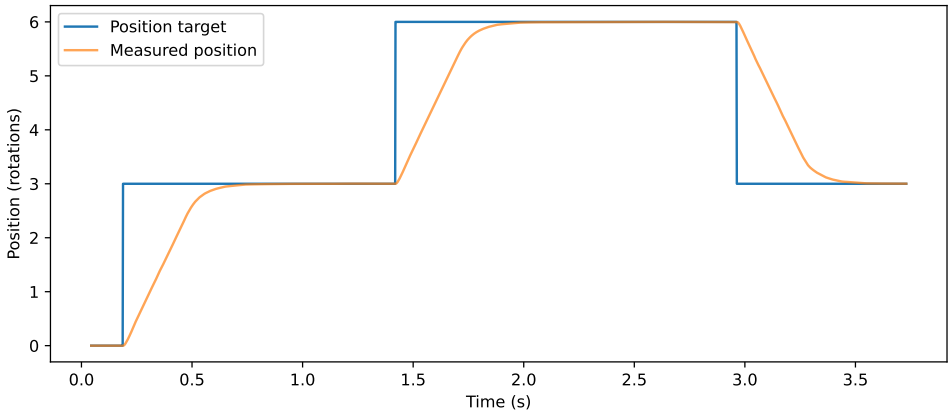

Figure 4.3: Position control

non-zero derivative gain of the PID controller was used to damp the system response and reduce overshoot during rapid position changes.

The results confirm correct functionality of the cascaded position, velocity, and current control loops.

4.2 Testing of client communication in daisy-chain mode

To demonstrate a practical use case of the firmware, a simple mirror-mode example was implemented in `pytools/mirror_mode.py`. The example uses two uFOC drivers, each connected to an iPower GM4108 motor.

The motors are connected in a daisy-chain CAN configuration. Each firmware instance is compiled with a different CAN ID, and one of the drivers is connected to a computer through a Makerbase CANable interface.

The Python script sets one motor to `NO_CONTROL` mode and the other motor to `POSITION_CONTROL` mode. In a continuous loop, the Python client reads the position of the first motor and sets the target position of the second motor to the same value. As a result, when the user manually moves the first motor, the second motor mirrors its motion.

This demonstration verifies the functionality of the CAN communication interface, runtime control mode switching, sensor feedback reading, and target position updating from the Python client. It also demonstrates that multiple uFOC drivers can share a single CAN bus in a daisy-chain configuration.

add video reference

5 | Future improvements

Encoder prediction for faster FOC loop

Encoder excentricity calibration

Developer feedback

Anti cogging

Multi motor path synchronization Dirrect can communication between devices - no client side

Combined commands

IMU support

6 | Conclusion

6.1 Future work

References

[1] Vladislav Bida, Dmitriy Samokhvalov, and Fuad Al-Mahturi. Pmsm vector control techniques — a survey. pages 577–581, 01 2018.

[2] Rofiq Cahyo, Aris Triwiyatno, and Munawar Riyadi. Field oriented control implementation on bldc motor controller with pi and svpwm using stm32f103c8t6. *Journal of Physics: Conference Series*, 2622:012025, 10 2023. [Online; accessed 15-May-2026].

[3] Corporate Finance Institute. Exponentially weighted moving average (ewma), 2024. [Online; accessed 2026-05-16].

[4] iFlight. ipower gm4108h-120t gimbal motor, 2026. [Online; accessed 16-May-2026].

[5] Benjamin Katz. *A low cost modular actuator for dynamic robots*. PhD thesis, 01 2018. [Online; accessed 15-May-2026].

[6] Gabriele Magnani, Daniele Cattaneo, Michele Chiari, and Giovanni Agosta. The Impact of Precision Tuning on Embedded Systems Performance: A Case Study on Field-Oriented Control. In João Bispo, Stefano Cherubin, and José Flich, editors, *12th Workshop on Parallel Programming and Run-Time Management Techniques for Many-core Architectures and 10th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2021)*, volume 88 of *Open Access Series in Informatics (OASIcs)*, pages 3:1–3:13, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. [Online; accessed 15-May-2026].

[7] Microchip Technology. Fundamentals of field-oriented control, 2024. Accessed: 2026-05-17.

[8] Microchip Technology Inc. Space vector pulse width modulation, 2024. [Online; accessed 16-May-2026].

[9] mjbots. moteus-c1 brushless servo controller, 2026. [Online; accessed 15-May-2026].

[10] ODrive Robotics. Odrive micro, 2026. [Online; accessed 15-May-2026].

[11] python-can project. python-can documentation. https://python-can.readthedocs.io/en/stable/, 2026. [Online; accessed 17-May-2026].

[12] Shanghai MagnTek Microelectronics Inc. *21-Bit High Accuracy Magnetic Angle Encoder IC*, 12 2022. [Online; accessed 15-May-2026].

[13] SimpleFOC. Field oriented control algorithm brief theory, 2024. Accessed: 2026-05-18.

[14] Antun Skuric, Hasan Sinan Bank, Richard Unger, Owen Williams, and David González-Reyes. Simplefoc: A field oriented control (foc) library for controlling brushless direct current (bldc) and stepper motors. *Journal of Open Source Software*, 7(74):4232, 2022. [Online; accessed 15-May-2026].

[15] Bc. Šimon Pecháček. ufoc hardware, 2025.